```
from pathlib import Path

GLOSSARY_SFT = Path("/content/glossary_sft.jsonl")

print("exists:", GLOSSARY_SFT.exists())
print("size:", GLOSSARY_SFT.stat().st_size if GLOSSARY_SFT.exists() else None)
```

```
exists: True
size: 48374
```

第2回 演習: LoRA で回答形式を学習し、学習前後を比較する

架空の講座 FAQ を題材に、小さな指示チューニング済みモデルを LoRA で短時間学習し、学習の前後で **何が変わり、何が変わらないか** を比べます。評価は **形式遵守率**（`answer` と `needs_review` の JSON を守れたか）と **内容正解率** に分けます。

この演習で確かめること

1. プロンプトで形式を指示しても、小さなモデルは守れないことがある（セクション 3、学習前）。
2. LoRA で数分学習すると、形式はほぼ守れるようになる（セクション 5、学習後）。
3. 形式が整っても、FAQ に無いことを聞かれたときの判断（確認を求めるか、断定してしまうか）までは保証されない（セクション 5、6）。
4. 発展: 同じモデルを 4bit 量子化で読み込むと、学習しなくても出力が変わる（セクション 8）。

進め方

セクション	内容	所要時間の目安（T4）
0〜3	環境、設定、データ生成、モデル読み込み、学習前評価	約 5 分
4〜7	学習、学習後評価、知識質問、比較表と提出	約 10 分（学習 4〜5 分を含む）
8（任意）	4bit 量子化で読み込んで比べる	約 2 分

授業前に 0〜3 を実行して環境が動くことを確かめておくと、当日の詰まりが減ります（任意）。Colab はセッションが切れると変数が消えるので、その場合は当日あらためて 0 から実行します。

使うもの: Google Colab（T4 GPU、無料枠）。モデルは `Qwen/Qwen3.5-0.8B`（Apache-2.0）。データはこのノートブックが生成する架空データで、実在の締切や URL は含みません。

読み方: 各セクションの説明に「やること」「見るところ」「うまくいかないとき」を書いています。セルは上から順に実行してください。途中のセルを飛ばすと、後のセルで変数が見つからないエラーになります。

版: 1.1（2026-10-18）

0. 環境の確認とライブラリの導入

やること: GPU が割り当てられているかを確認し、ライブラリを動作確認済みの版に固定して導入します。

見るところ

- 1 つ目のセルに `Tesla T4` と表示されること。
- 2 つ目のセルの最後に、版の一覧（`transformers: 5.18.0` など）が表示されること。導入は 1 分ほどかかります。

うまくいかないとき

- 「GPU が見つかりません」と出る: メニューの「ランタイム」→「ランタイムのタイプを変更」→「T4 GPU」を選び、セルを実行し直します。混雑で割り当てられないときは、時間をおいて試します。
- pip の赤い `ERROR: pip's dependency resolver ...`（`cudf` や `pyarrow` に関するもの）は、Colab に最初から入っている別のパッケージとの食い違いの警告です。この演習には影響しないので、そのまま進めます。

```
import subprocess
try:
    gpu_info = subprocess.run(["nvidia-smi", "--query-gpu=name,memory.total", "--format=csv"], capture_output=True, text=True).stdout
except FileNotFoundError:   # CPU ランタイムには nvidia-smi が無い
    gpu_info = ""
print(gpu_info or "GPU が見つかりません。ランタイム → ランタイムのタイプを変更 → T4 GPU を選んでください。")
```

```
name, memory.total [MiB]
Tesla T4, 15360 MiB
```

```
# Colab の T4（Python 3.13、torch 2.11.0+cu130）で動作確認した版。torch は Colab 同梱のものをそのまま使う
PINS = {
    "transformers": "5.18.0",   # Qwen3.5 は transformers 5 系が必要
    "peft": "0.21.1",           # LoRA の実装
    "trl": "1.14.2",            # 学習ループ（SFTTrainer）
    "accelerate": "1.15.0",
```

```
    "accelerate": "1.15.0",
    "datasets": "5.1.0",
}
spec = " ".join(f'"{k}=={v}"' for k, v in PINS.items())
%pip uninstall -y -q torchao  # Colab 同梱の torchao (0.10) は peft 0.21 と非互換で、学習時に ImportError になる。使わないので外す
%pip install -q {spec}

# cudf-cu13 と pyarrow の依存警告が ERROR 表示されるが、cudf は使わないので無視してよい
import transformers, peft, trl, accelerate, datasets, torch
print({m.__name__: m.__version__ for m in [transformers, peft, trl, accelerate, datasets, torch]})
```

```
WARNING: Skipping torchao as it is not installed.
{'transformers': '5.18.0', 'peft': '0.21.1', 'trl': '1.14.2', 'accelerate': '1.15.0', 'datasets': '5.1.0', 'torch': '2.11.0+cu130'}
```

1. 設定

やること: モデル名、学習のエポック数（データを何周するか）などを 1 つの辞書 `CONFIG` にまとめます。この辞書はセクション 7 の提出用ファイルにそのまま記録されるので、「どの条件で測ったか」が後から分かります。

- `revision` を固定しているのは、いつ実行しても同じ重みを使うためです。学習前後の比較や、他の人との比較で条件が揃います。
- `num_train_epochs: 20` は、学習データ（14 件）を 20 周する設定です。1 周が 2 ステップなので合計 40 ステップで、T4 で学習が 4〜5 分に収まります。
- `gen: {"do_sample": False}` は貪欲生成です。学習前後で生成条件を揃え、揺れの影響を除きます。

`USE_DRIVE` を `True` にすると Google Drive をマウントし、学習した adapter と結果ファイルを Drive にも保存します。セッションが切れても結果を残したいときに使います。1 回のセッションで完結させるなら `False` のままで構いません。

見るところ: 設定の JSON が表示されます。

```
from pathlib import Path
import os, json, time

CONFIG = {
    "base_model": "Qwen/Qwen3.5-0.8B",   # 小さな指示チューニング済みモデル（Apache-2.0）。量子化なしで T4 に載る大きさ
    "revision": "2fc06364715b967f1860aea9cf38778875588b17",  # いつ実行しても同じ重みを使うために固定
    "seed": 42,
    "num_train_epochs": 20,  # 学習データ（14 件）を何周学習するか。1 周 2 ステップなので合計 40 ステップ（T4 で 4〜5 分）
    "warmup_steps": 6,
    "lr": 2e-4,
    "lora_r": 16,            # LoRA の行列の次元（小さいほど学習するパラメータが少ない）
    "lora_alpha": 32,
    "max_seq_len": 512,
    "max_new_tokens": 96,
    "gen": {"do_sample": False},   # 評価は貪欲生成で固定（前後比較の条件を揃える）
    "work_dir": "/content/qlora_lab",                    # 結果ファイルの保存先（左のファイル一覧から見られる）
    "drive_dir": "/content/drive/MyDrive/aie2026/qlora_lab",  # USE_DRIVE=True のときの保存先
}
WORK = Path(CONFIG["work_dir"]); WORK.mkdir(parents=True, exist_ok=True)

USE_DRIVE = False  # True にすると Google Drive をマウントして adapter と出力を保存する
if USE_DRIVE:
    from google.colab import drive
    drive.mount("/content/drive")
    Path(CONFIG["drive_dir"]).mkdir(parents=True, exist_ok=True)
print(json.dumps(CONFIG, ensure_ascii=False, indent=2))
```

```
{
  "base_model": "Qwen/Qwen3.5-0.8B",
  "revision": "2fc06364715b967f1860aea9cf38778875588b17",
  "seed": 42,
  "num_train_epochs": 20,
  "warmup_steps": 6,
  "lr": 0.0002,
  "lora_r": 16,
  "lora_alpha": 32,
  "max_seq_len": 512,
  "max_new_tokens": 96,
  "gen": {
    "do_sample": false
  },
  "work_dir": "/content/qlora_lab",
  "drive_dir": "/content/drive/MyDrive/aie2026/qlora_lab"
}
```

2. 架空 FAQ と教師データの生成

やること: 5 項目の架空 FAQ から、学習用（train）、検証用（valid）、評価用（test）のデータを作ります。

モデルへの入力は「FAQ ＋ 質問」、望ましい出力は `{"answer": <文字列>, "needs_review": <真偽値>}` の JSON 1 行です。

- FAQ に根拠がある質問には、FAQ の文で答え、`needs_review: false` にする。
- FAQ に無い質問には、「この FAQ には記載がありません。運営に確認が必要です。」と答え、`needs_review: true` にする。

データの分け方

- 同じ FAQ 項目を聞く質問の言い換えを、**テンプレート単位**で train / valid / test に振り分けます。学習で見た文がそのまま test に出ないようにするためです。
- test は 21 問（根拠あり 15、根拠なし 6）です。件数を確保するため、test にだけ使う質問を別に用意しています。
- 正解（`needs_review` の値と、答えに含まれるべき語句）は `gold` に分けて持ち、モデルには渡しません。
- train は 14 件（根拠あり 10、根拠なし 4）、valid は 7 件です。FAQ が 5 項目しかないので学習データは少なく、同じ例を何回も学習することになります。何回学習するかは、データを複製して増やすのではなく、セクション 4 のエポック数（`num_train_epochs`）で決めます。データのファイルには実例だけを入れ、学習量は学習の設定で持つのが基本です。

見るところ: `train=14 valid=7 test=21 gold=21` と、学習例の 1 件（質問と、望ましい出力）が表示されます。質問が train / valid / test のどれかで重複していると、ここでエラーになります。

```
import json
from pathlib import Path

GLOSSARY_SFT = Path("/content/glossary_sft.jsonl")

# ----------------------------
# 1. SFT学習データの読み込み
# ----------------------------
with GLOSSARY_SFT.open(encoding="utf-8") as f:
    glossary_records = [
        json.loads(line)
        for line in f
        if line.strip()
    ]

print("SFT records:", len(glossary_records))


# ----------------------------
# 2. Gold
# ----------------------------
GOLD_DEFINITIONS = {
    "LoRA":
        "事前学習済みの重みは固定したまま、重みの更新分を2つの小さな行列の積（低ランク行列）で表し、その小さな行列だけを学習するPEFT手法

    "QLoRA":
        "ベースモデルの重みを4ビットなどに量子化してメモリを節約したうえで、LoRAで学習する手法。より大きなモデルを小さなGPUで扱える。本

    "ファインチューニング":
        "事前学習済みモデルを、特定のタスクや形式に合ったデータで追加学習させること。出力の形式や口調、特定ドメインの振る舞いを身につけ

    "ハルシネーション":
        "LLMが事実と異なる内容や存在しない情報を、もっともらしく生成してしまう現象。モデルが知らないこと（最新情報、組織や講座に固有の情

    "MCP（Model Context Protocol）":
        "AIアプリと外部のツールやデータソースをつなぐためのオープンな標準プロトコル。MCPサーバーとして公開したツールは、対応する様々なク
}


# ----------------------------
# 3. 評価問題
# ----------------------------
TEST = [
    # ======================================
    # Exact
    # 用語集に存在する用語を直接質問
    # ======================================
    {
        "id": "E1",
        "category": "exact",
        "term": "LoRA",
        "question": "LoRAとは何ですか？",
    },
    {
        "id": "E2",
        "category": "exact",
        "term": "QLoRA",
        "question": "QLoRAとは何ですか？",
    },
    {
        "id": "E3",
        "category": "exact",
        "term": "ファインチューニング",
```

```
    "question": "ファインチューニングとは何ですか？",
},
{
    "id": "E4",
    "category": "exact",
    "term": "ハルシネーション",
    "question": "ハルシネーションとは何ですか？",
},
{
    "id": "E5",
    "category": "exact",
    "term": "MCP (Model Context Protocol) ",
    "question": "MCP (Model Context Protocol) とは何ですか？",
},

# ======================================
# Paraphrase
# 学習時とは異なる言い回しで質問
# ======================================
{
    "id": "P1",
    "category": "paraphrase",
    "term": "LoRA",
    "question": "LoRAは、事前学習済みモデルをどのように効率よく追加学習する仕組みですか？",
},
{
    "id": "P2",
    "category": "paraphrase",
    "term": "QLoRA",
    "question": "QLoRAでは、モデルのメモリ使用量を抑えながらどのようにLoRA学習を行いますか？",
},
{
    "id": "P3",
    "category": "paraphrase",
    "term": "ファインチューニング",
    "question": "事前学習済みモデルを特定の用途に合わせて調整する方法を説明してください。",
},
{
    "id": "P4",
    "category": "paraphrase",
    "term": "ハルシネーション",
    "question": "LLMが事実ではない情報をもっともらしく生成する現象について説明してください。",
},
{
    "id": "P5",
    "category": "paraphrase",
    "term": "MCP (Model Context Protocol) ",
    "question": "AIアプリと外部ツールやデータソースを共通の方法で接続する仕組みについて説明してください。",
},

# ======================================
# Unknown
# 用語集学習データに含まれない用語を質問
# ======================================
{
    "id": "U1",
    "category": "unknown",
    "term": "Knowledge Graph",
    "question": "Knowledge Graphとは何ですか？",
},
{
    "id": "U2",
    "category": "unknown",
    "term": "Embedding",
    "question": "Embeddingとは何ですか？",
},
{
    "id": "U3",
    "category": "unknown",
    "term": "ベクトルデータベース",
    "question": "ベクトルデータベースとは何ですか？",
},
{
    "id": "U4",
    "category": "unknown",
    "term": "Chain-of-Thought",
    "question": "Chain-of-Thoughtとは何ですか？",
},
{
    "id": "U5",
    "category": "unknown",
    "term": "RLHF",
```

```
        "question": "RLHFとは何ですか？",
    },
]


# ----------------------------
# 4. 評価データ件数の確認
# ----------------------------
print("TEST:", len(TEST))

for category in ["exact", "paraphrase", "unknown"]:
    print(
        category,
        sum(x["category"] == category for x in TEST)
    )


# ----------------------------
# 5. Unknown用語が学習データ本文に
#    含まれていないことを確認
# ----------------------------
unknown_terms = [
    x["term"]
    for x in TEST
    if x["category"] == "unknown"
]

training_text = "¥n".join(
    msg["content"]
    for record in glossary_records
    for msg in record["messages"]
)

print("¥nUnknown terms check:")

for term in unknown_terms:
    print(term, "->", term in training_text)
```

```
SFT records: 116
TEST: 15
exact 5
paraphrase 5
unknown 5

Unknown terms check:
Knowledge Graph -> False
Embedding -> False
ベクトルデータベース -> False
Chain-of-Thought -> False
RLHF -> False
```

3. モデルの読み込みと学習前評価

やること: 基盤モデルを量子化なし（32bit）で読み込み、test の 21 問に答えさせて採点します。

プロンプト（`SYSTEM`）には「必ず JSON 1 行で、`answer` と `needs_review` を返す」と書いてあります。つまり **形式は指示済み** です。このモデルがその指示を守れるかどうかが、学習前の確認点です。

採点のしかた（セクション 5 でも同じ関数を使います）

- **形式**: JSON として読め、キーが `answer` と `needs_review` のちょうど 2 つで、型が文字列と真偽値であること。
- **内容**: `needs_review` が正解と一致し、かつ正解の語句（例: 「9 月 30 日」と「23:59」）が `answer` に含まれること。

内容の判定には `needs_review` の一致が必要です。そのため、キーが欠けている出力は、文面が正しくても内容は × になります。文面そのものは、表で自分の目で確かめてください。

`Qwen/Qwen3.5-0.8B` は画像も扱えるモデルですが、`AutoModelForCausalLM` で読み込むとテキスト部分だけが使われます。

見るところ

- 1 つ目のセル: `重みのメモリ(GB): 3.01` のように表示されます。モデルのダウンロード（約 1.75GB）に 1 分ほどかかります。
- 2 つ目のセル: 1 行の要約（`format_rate`、`content_rate` など）と、21 問の出力の表。評価には 1 分前後かかります。学習前は、多くの出力で `needs_review` が欠けているはずです。

出ても問題ない警告

- `You are sending unauthenticated requests to the HF Hub`: Hugging Face にログインせずにモデルを取りに行っている、という案内です。公開モデルはログインしなくてもダウンロードできます。ログインが要るのは次の場合で、モデルの大きさでは決まりません。
    - 利用規約への同意が必要なモデル（gated model。Llama や Gemma など）や、非公開のリポジトリを使うとき。

  - 自分のモデルやデータを Hub にアップロードするとき。
  - 短時間に大量のリクエストを送り、回数の上限に当たるとき。上限はログインしていないと IP アドレスごとに 5 分あたり 3,000 回のファイル取得（2025 年 9 月時点の公表値。Hub の rate limits）で、ログインすると上がります。

  このノートブックのモデルは公開されていて同意も不要です。読み込みで送るリクエストは多くても数十回なので、上限には届きません。もし上限に当たって `429 Too Many Requests` が出ても、ライブラリが待ち時間を読み取って自動で再試行します。ログインが必要になったときは、Colab の左側の鍵アイコン（シークレット）に Hugging Face のトークンを `HF_TOKEN` という名前で登録し、このノートブックからのアクセスを許可すると、自動で使われます。

- `... is falling back to its reference PyTorch implementation ...`: このモデルの一部の層（線形注意）に対する高速な実装が入っていないので、遅いが正しい実装で動くという案内です。4 行ほど出ます。

ここで読んでおくこと: 表を見て、「どのキーが欠けているか」「文面は FAQ と合っているか」を確認します。この表はセクション 5 で学習後の出力と並べて見ます。出力の原文は `/content/qlora_lab/eval_before.json` にも保存されています（左のファイル一覧から開けます）。

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM

DEVICE = "cuda" if torch.cuda.is_available() else ("mps" if torch.backends.mps.is_available() else "cpu")
tok = AutoTokenizer.from_pretrained(CONFIG["base_model"], revision=CONFIG["revision"])
if tok.pad_token is None: tok.pad_token = tok.eos_token
# 量子化なし。重みは 32bit のまま保持する（dtype を省くと配布時の精度で読み込まれるので明示する）
model = AutoModelForCausalLM.from_pretrained(CONFIG["base_model"], revision=CONFIG["revision"], dtype=torch.float32).to(DEVICE)
model.config.use_cache = True
BASE_MEMORY_GB = round(model.get_memory_footprint() / 1e9, 2)
print("loaded:", CONFIG["base_model"], "| device:", DEVICE, "| 重みのメモリ(GB):", BASE_MEMORY_GB)

# 形式はプロンプトで指示している。モデルがこれを守れるかが学習前の確認点
SYSTEM = "あなたは講座 FAQ の回答係です。与えられた FAQ だけを根拠に、必ず JSON 1 行で答えてください。形式: {¥"answer¥": 文字列, ¥"need

def build_messages(example):
    user = f"FAQ:¥n{example['faq']}¥n¥n質問: {example['question']}"
    return [{"role": "system", "content": SYSTEM}, {"role": "user", "content": user}]

def generate(model, example):
    # enable_thinking=False: 思考ブロックを空にして回答だけを生成させる（モデルを替えても条件が揃うように明示）
    prompt = tok.apply_chat_template(build_messages(example), tokenize=False, add_generation_prompt=True, enable_thinking=False)
    inputs = tok(prompt, return_tensors="pt").to(model.device)
    with torch.no_grad():
        out = model.generate(**inputs, max_new_tokens=CONFIG["max_new_tokens"], pad_token_id=tok.pad_token_id, **CONFIG["gen"])
    return tok.decode(out[0][inputs["input_ids"].shape[1]:], skip_special_tokens=True).strip()
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable hig
config.json: 100%  2.91k/2.91k [00:00<00:00, 203kB/s]
tokenizer_config.json: 100%  16.7k/16.7k [00:00<00:00, 521kB/s]
tokenizer.json: reconstructing file: 100%  12.8MB / 12.8MB, 1.26MB/s
tokenizer.json: downloading bytes:  5.68MB, 556kB/s
chat_template.jinja: 100%  7.75k/7.75k [00:00<00:00, 377kB/s]
model.safetensors.index.json: 100%  50.9k/50.9k [00:00<00:00, 1.62MB/s]
model.safetensors-00001-of-  1.75GB / 1.75GB, 103MB/s
00001.safeten(…): reconstructing file: 100%
model.safetensors-00001-of-00001.safeten(…): downloading bytes:  1.52GB, 64.1MB/s
Loading weights: 100%  320/320 [00:02<00:00, 175.10it/s]
loaded: Qwen/Qwen3-5-0.8B | device: cuda | 重みのメモリ(GB): 2.01
```

```
print("model:", type(model).__name__)
print("tokenizer:", type(tok).__name__)
print("device:", model.device)
```

```
model: Qwen3_5ForCausalLM
tokenizer: Qwen3_5Tokenizer
device: cuda:0
```

```
import time
import json
import pandas as pd
import torch

def generate_glossary_answer(model, question, max_new_tokens=256):
    messages = [
        {"role": "user", "content": question}
```

```
    ]

    inputs = tok.apply_chat_template(
        messages,
        add_generation_prompt=True,
        return_tensors="pt",
        return_dict=True,
    ).to(model.device)

    t0 = time.time()

    with torch.no_grad():
        outputs = model.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
            do_sample=False,
        )

    elapsed = time.time() - t0

    # 入力部分を除き、生成された回答部分だけを取得
    generated = outputs[0][inputs["input_ids"].shape[1]:]

    answer = tok.decode(
        generated,
        skip_special_tokens=True,
    ).strip()

    return answer, elapsed


# ------------------------------
# 学習前評価
# ------------------------------
before_rows = []

for item in TEST:
    answer, elapsed = generate_glossary_answer(
        model,
        item["question"],
    )

    before_rows.append({
        "id": item["id"],
        "category": item["category"],
        "term": item["term"],
        "question": item["question"],
        "answer_before": answer,
        "seconds_before": round(elapsed, 3),
    })

before_df = pd.DataFrame(before_rows)

display(before_df)


# ------------------------------
# 学習前結果を保存
# ------------------------------
BEFORE_PATH = WORK / "glossary_eval_before.json"

BEFORE_PATH.write_text(
    json.dumps(
        before_rows,
        ensure_ascii=False,
        indent=2,
    ),
    encoding="utf-8",
)

print("saved:", BEFORE_PATH)
```

```
[transformers] `causal_conv1d_fn` is falling back to its reference PyTorch implementation because `causal_conv1d` is not installed. This
[transformers] `chunk_gated_delta_rule` is falling back to its reference PyTorch implementation because `flash-linear-attention` is not
[transformers] `causal_conv1d_update` is falling back to its reference PyTorch implementation because `causal_conv1d` is not installed.
[transformers] `fused_recurrent_gated_delta_rule` is falling back to its reference PyTorch implementation because `flash-linear-attentio
```

	id	category	term	question	answer_before	seconds_before
0	E1	exact	LoRA	LoRAとは何ですか？	**LoRA** は、大規模言語モデル（LLM）のトレーニングにおいて非常に重要な技術の一つ...	16.164
1	E2	exact	QLoRA	QLoRAとは何ですか？	**QLoRA** は、**Qwen3.5** という大規模言語モデル（LLM）の一種です。...	11.479
2	E3	exact	ファインチューニング	ファインチューニングとは何ですか？	**ファインチューニング**（Fine Tuning）は、ソフトウェアやシステムにおいて、ユ...	11.285
3	E4	exact	ハルシネーション	ハルシネーションとは何ですか？	**ハルシネーション**（Hulshinēshon）は、物理学、特に**量子力学**において...	11.293
4	E5	exact	MCP（Model Context Protocol）	MCP（Model Context Protocol）とは何ですか？	**MCP（Model Context Protocol）** は、大規模言語モデル（LLM...	11.347
5	P1	paraphrase	LoRA	LoRAは、事前学習済みモデルをどのように効率よく追加学習する仕組みですか？	LoRA（Low-Rank Adaptation）は、事前学習済みモデル（通常は大規模な T...	11.183
6	P2	paraphrase	QLoRA	QLoRAでは、モデルのメモリ使用量を抑えながらどのようにLoRA学習を行いますか？	QLoRA（Quantized LoRA）は、LoRA（低秩近似）の学習を**「メモリ使用量...	11.307
7	P3	paraphrase	ファインチューニング	事前学習済みモデルを特定の用途に合わせて調整する方法を説明してください。	事前学習済みモデル（Pre-trained Models）は、大規模言語モデル（LLM）や画...	11.394
8	P4	paraphrase	ハルシネーション	LLMが事実ではない情報をもっともらしく生成する現象について説明してください。	LLM（大規模言語モデル）が事実ではない情報をもっともらしく生成する現象は、**「生成誤解（...	11.233
9	P5	paraphrase	MCP（Model Context Protocol）	AIアプリと外部ツールやデータソースを共通の方法で接続する仕組みについて説明してください。	AI アプリと外部ツール（API）、データソース（API、データベース、Web サイトなど）...	11.262
10	U1	unknown	Knowledge Graph	Knowledge Graphとは何ですか？	**知識グラフ（Knowledge Graph, KG）** は、データが「知識」として整理...	11.273
11	U2	unknown	Embedding	Embeddingとは何ですか？	**Embedding**（エミブディング）は、データや画像などの情報を「埋め込み（埋め込む...	16.590
12	U3	unknown	ベクトルデータベース	ベクトルデータベースとは何ですか？	**ベクトルデータベース**（Vector Database）は、データが「ベクトル（数値列...	11.245
13	U4	unknown	Chain-of-Thought	Chain-of-Thoughtとは何ですか？	**Chain-of-Thought (CoT)** は、AI の学習や判断において、**「...	11.211

次のステップ: before_df を使用してコードを生成 New interactive sheet

```python
import re
import pandas as pd
pd.set_option("display.max_colwidth", None)  # 出力の原文を省略せずに表示する

def check_format(text: str):
    '''JSON として読め、必須キーと型が合うか。失敗理由も返す'''
    try:
        obj = json.loads(text)
    except Exception as e:
        return False, f"json_error: {type(e).__name__}", None
    if not isinstance(obj, dict): return False, "not_object", None
    if set(obj) - {"answer", "needs_review"}: return False, "extra_keys", obj
    if "answer" not in obj or "needs_review" not in obj: return False, "missing_key", obj
    if not isinstance(obj["answer"], str) or not isinstance(obj["needs_review"], bool): return False, "wrong_type", obj
    return True, "", obj

def check_content(obj, gold_entry):
    '''内容の合否: needs_review が正解と一致し、必須語句を answer に含む'''
    if obj is None: return False
    if obj.get("needs_review") != gold_entry["needs_review"]: return False
    return all(w in obj.get("answer", "") for w in gold_entry["must_include"])

def mark(ok: bool) -> str:
    return "○" if ok else "×"

def evaluate(model, label: str):
    '''test の 21 問を生成して採点し、要約と質問ごとの結果を返す。原文は eval_<label>.json に保存する'''
```

```
    rows = []
    t0 = time.time()
    for ex in test:
        raw = generate(model, ex)
        ok_fmt, reason, obj = check_format(raw)
        # 形式が壊れていても JSON らしい部分が読めるなら、内容だけは別に採点する
        loose = obj
        if not ok_fmt:
            m = re.search(r"¥{.*¥}", raw, re.S)
            try: loose = json.loads(m.group(0)) if m else None
            except Exception: loose = None
        ok_content = check_content(loose if isinstance(loose, dict) else None, gold[ex["question"]])
        rows.append({"question": ex["question"], "raw": raw, "format_ok": ok_fmt, "format_reason": reason,
                     "content_ok": ok_content, "both_ok": ok_fmt and ok_content, "gold_needs_review": gold[ex["question"]]["needs_review"
    n = len(rows)
    summary = {"label": label, "n": n,
               "format_rate": sum(r["format_ok"] for r in rows)/n,
               "content_rate": sum(r["content_ok"] for r in rows)/n,
               "both_rate": sum(r["both_ok"] for r in rows)/n,
               "seconds": round(time.time()-t0, 1)}
    (WORK / f"eval_{label}.json").write_text(json.dumps({"summary": summary, "rows": rows}, ensure_ascii=False, indent=2), encoding="utf-
    print(json.dumps(summary, ensure_ascii=False))
    return summary, rows

before_summary, before_rows = evaluate(model, "before")
# 質問ごとの出力。「正解の needs_review」と見比べ、どのキーが欠けているか、文面は FAQ と合っているかを読む
display(pd.DataFrame([{"質問": r["question"], "正解の needs_review": r["gold_needs_review"], "学習前の出力": r["raw"],
                       "形式": mark(r["format_ok"]), "形式の失敗理由": r["format_reason"], "内容": mark(r["content_ok"])} for r in before
```

4. LoRA 学習（T4 で 4〜5 分）

LoRA と QLoRA の違い

- **LoRA**: 基盤モデルの重みは固定し、一部の層に追加した小さな行列（adapter）だけを学習します。
- **QLoRA**: 基盤モデルの重みを 4bit に量子化して保持し、その上で同じように adapter だけを学習します。adapter 自体は量子化しません。

QLoRA の効果は、基盤モデルの重みを保持するメモリが減ることです。16bit の約 4 分の 1 になるので、そのままでは GPU に載らない大きさのモデルでも学習できます（70 億パラメータなら 16bit で約 14GB、4bit で約 3.5GB）。

観点	LoRA（量子化なし）	QLoRA（4bit）
重みのメモリ	大きい	小さい（16bit の約 1/4）
基盤モデルの出力	元のまま	量子化の誤差で変わることがある。影響の大きさはモデルによって違う
学習の速さ	基準	速くはならない。4bit の重みを計算のたびに戻すので、遅くなることが多い
依存ライブラリ	peft	peft に加えて bitsandbytes（CUDA の GPU が前提）

QLoRA を実装するときの注意:

- 読み込み時に `BitsAndBytesConfig(load_in_4bit=True, ...)` を渡し、LoRA を付ける前に `prepare_model_for_kbit_training(model)` を通します。
- 保持精度（4bit）と計算精度（`bnb_4bit_compute_dtype`。16bit など）は別の設定です。
- 学習前後の比較は、同じ量子化条件の基盤モデルで行います。学習前を量子化なし、学習後を 4bit で測ると、量子化の影響と学習の影響が混ざります。
- adapter は学習時の基盤モデルに合わせて作られます。配布や再利用のときは、基盤モデルの ID・revision と量子化設定を一緒に記録します。
- 4bit での学習が勧められないモデルもあります。Qwen3.5 は量子化の誤差が通常より大きいとして、4bit 学習を勧めていない学習ライブラリのガイドがあります。

この演習のモデルは量子化なしでも T4 に載るので、LoRA で学習します。モデルが GPU に載るなら LoRA、載らないなら QLoRA かより小さいモデル、という順で考えます。4bit で読み込んだ場合のメモリと出力の違いは、セクション 8（発展）で確かめられます。

学習の実行

やること: train の 14 件を chat 形式（system / user / assistant）に直し、LoRA を付けて 20 エポック（40 ステップ）学習します。学習した adapter は `adapter/` に保存します。

- `target_modules` は LoRA を差し込む層です。注意機構の射影（`q_proj` など）と MLP の射影（`gate_proj` など）を指定しています。このモデルは 24 層のうち 18 層が線形注意で、`q_proj` などを持つのは残りの 6 層だけです。MLP は全層にあります。
- バッチ 4 × 勾配累積 2 = 1 ステップ 8 件。train は 14 件なので 1 エポックは 2 ステップ（8 件と 6 件）で、20 エポックで 40 ステップです。各例をちょうど 20 回学習します。学習率の warmup と減衰（cosine）はこの 40 ステップに対して効きます。
- 学習後のモデルは変数 `tuned` に入ります。LoRA は基盤モデルの層に差し込まれるので、元の変数 `model` で生成しても adapter 込みの出力になります。学習前の出力が必要なときは、セクション 6 のように adapter を一時的に無効にします。

見るところ

- `trainable params: ... || all params: ... || trainable%: ...`: 学習するパラメータが全体の数 % だけであることが LoRA の特徴です。
- `Training Loss` の表（10 ステップごと）: 1.5 前後から始まり、40 ステップで 0.01〜0.02 まで下がります。下がらないまま終わった場合は、設定を変えていないか確かめてください。
- `学習時間: ... 秒 | ステップ数: 40 | エポック: 20.0`: T4 で 240〜290 秒です。ステップ数とエポックは、学習データ 14 件 × 20 周が上の計算どおりに実行されたことの確認です。

出ても問題ない警告: `The tokenizer has new PAD/BOS/EOS tokens that differ from the model config ...` は、終端トークンの設定を tokenizer に合わせたという案内です。

うまくいかないとき: 学習が途中で止まった、または学習できなかった場合は、下の 4b を読んでください。

```
from peft import LoraConfig
from trl import SFTTrainer, SFTConfig
from datasets import Dataset
import time

# ----------------------------------------------------
# 用語集SFTデータ
# make_sft_dataset.py が生成した各レコードは
# すでに messages 形式なので、そのまま Dataset にする
# ----------------------------------------------------
train_ds = Dataset.from_list(glossary_records)

print("train records:", len(train_ds))
print("first record:")
print(train_ds[0])


# ----------------------------------------------------
# LoRA設定
# ----------------------------------------------------
model.config.use_cache = False

# Qwen3.5 は 24 層のうち 18 層が線形注意で、
# q/k/v/o_proj を持つのは残りの 6 層。
# gate/up/down_proj (MLP) は全層にある
lora = LoraConfig(
    r=CONFIG["lora_r"],
    lora_alpha=CONFIG["lora_alpha"],
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM",
    target_modules=[
        "q_proj",
        "k_proj",
        "v_proj",
        "o_proj",
        "gate_proj",
        "up_proj",
        "down_proj",
    ],
)

USE_AMP = False

sft = SFTConfig(
    output_dir=str(WORK / "ckpt"),
    num_train_epochs=CONFIG["num_train_epochs"],
    per_device_train_batch_size=4,
    gradient_accumulation_steps=2,
    learning_rate=CONFIG["lr"],
    lr_scheduler_type="cosine",
    warmup_steps=CONFIG["warmup_steps"],
    logging_steps=10,
    save_steps=10,
    save_total_limit=2,
    fp16=USE_AMP,
    max_length=CONFIG["max_seq_len"],
    packing=False,
    report_to="none",
    seed=CONFIG["seed"],
)

trainer = SFTTrainer(
    model=model,
    args=sft,
    train_dataset=train_ds,
```

```
    peft_config=lora,
    processing_class=tok,
)

trainer.model.print_trainable_parameters()

t0 = time.time()

trainer.train()

training_seconds = time.time() - t0

print(
    f"学習時間: {training_seconds:.0f} 秒 | "
    f"ステップ数: {trainer.state.global_step} | "
    f"エポック: {trainer.state.epoch:.1f}"
)


# ----------------------------------------------------
# Adapter保存
# ----------------------------------------------------
ADAPTER_DIR = WORK / "glossary_adapter"

trainer.model.save_pretrained(ADAPTER_DIR)

if USE_DRIVE:
    import shutil

    shutil.copytree(
        ADAPTER_DIR,
        Path(CONFIG["drive_dir"]) / "glossary_adapter",
        dirs_exist_ok=True,
    )


# ----------------------------------------------------
# 学習済みモデル
# ----------------------------------------------------
tuned = trainer.model
tuned.config.use_cache = True

# dropoutを無効化して評価モードにする
_ = tuned.eval()

print("adapter saved:", ADAPTER_DIR)
```

```
train records: 116
first record:
{'messages': [{'role': 'user', 'content': '「AIエンジニアリング」とは何ですか？'}, {'role': 'assistant', 'content': '学習済みの基盤モデル
```

```
Tokenizing train dataset: 100%  116/116 [00:00<00:00, 711.11 examples/s]
Building labels for train dataset: 100%  116/116 [00:00<00:00, 3345.68 examples/s]
Truncating train dataset: 100%  116/116 [00:00<00:00, 3449.24 examples/s]
Dropping fully masked examples from train dataset: 100%  116/116 [00:00<00:00, 6020.71 examples/s]
```

```
[transformers] The tokenizer has new PAD/BOS/EOS tokens that differ from the model config and generation config. The model config and ge
trainable params: 6,389,760 || all params: 758,782,784 || trainable%: 0.8421
```

[300/300 11:41, Epoch 20/20]

Step	Training Loss
10	3.276617
20	2.213159
30	1.670747
40	0.961685
50	0.555070
60	0.339149
70	0.169443
80	0.149946
90	0.134071
100	0.113787
110	0.102804
120	0.106254
130	0.092151
140	0.091040
150	0.084064
160	0.077544
170	0.077250
180	0.081164
190	0.075279
200	0.074195
210	0.075184
220	0.073136
230	0.072939
240	0.072720
250	0.070116
260	0.070896
270	0.071449
280	0.069571
290	0.071499
300	0.068818

```
学習時間: 704 秒 | ステップ数: 300 | エポック: 20.0
adapter saved: /content/qlora_lab/glossary_adapter
```

```
import json
import pandas as pd

# ------------------------------
# 学習後評価
# ------------------------------
after_rows = []

for item in TEST:
    answer, elapsed = generate_glossary_answer(
        tuned,
        item["question"],
    )
```

```
    after_rows.append({
        "id": item["id"],
        "category": item["category"],
        "term": item["term"],
        "question": item["question"],
        "answer_after": answer,
        "seconds_after": round(elapsed, 3),
    })

after_df = pd.DataFrame(after_rows)

display(after_df)


# ----------------------------
# 学習後結果を保存
# ----------------------------
AFTER_PATH = WORK / "glossary_eval_after.json"

AFTER_PATH.write_text(
    json.dumps(
        after_rows,
        ensure_ascii=False,
        indent=2,
    ),
    encoding="utf-8",
)

print("saved:", AFTER_PATH)


# ----------------------------
# Before / After を結合
# ----------------------------
compare_df = before_df.merge(
    after_df,
    on=["id", "category", "term", "question"],
    how="inner",
)

display(
    compare_df[
        [
            "id",
            "category",
            "term",
            "question",
            "answer_before",
            "answer_after",
            "seconds_before",
            "seconds_after",
        ]
    ]
)
```

	id	category	term	question	answer_after	seconds_after
0	E1	exact	LoRA	LoRAとは何ですか？	事前学習済みの重みは固定したまま、重みの更新分を2つの小さな行列の積（低ランク行列）で表し、...	9.784
1	E2	exact	QLoRA	QLoRAとは何ですか？	ベースモデルの重みを4ビットなどに量子化してメモリを節約したうえで、LoRAで学習する手法。...	9.866
2	E3	exact	ファインチューニング	ファインチューニングとは何ですか？	事前学習済みモデルを、特定のタスクや形式に合ったデータで追加学習させること。出力の形式や口調...	8.916
3	E4	exact	ハルシネーション	ハルシネーションとは何ですか？	LLMが事実と異なる内容や存在しない情報を、もっともらしく生成してしまう現象。モデルが知らな...	7.652
4	E5	exact	MCP（Model Context Protocol）	MCP（Model Context Protocol）とは何ですか？	AIアプリと外部のツールやデータソースをつなぐためのオープンな標準プロトコル。MCPサーバー...	9.120
5	P1	paraphrase	LoRA	LoRAは、事前学習済みモデルをどのように効率よく追加学習する仕組みですか？	固定した低ランクモデル（例：10ビット）を学習するモデル（通常4ビット）。低ランクモデルは精...	7.108
6	P2	paraphrase	QLoRA	QLoRAでは、モデルのメモリ使用量を抑えながらどのようにLoRA学習を行いますか？	固定ビット数（例：4ビットでモデルを読み込む）でメモリを節約し、LoRAのrを大きくする組み...	11.076
7	P3	paraphrase	ファインチューニング	事前学習済みモデルを特定の用途に合わせて調整する方法を説明してください。	AIE2026の演習と宿題で、所轄のモデル（LoRA-100、Pi-5、Pi-1）を使い、そ...	10.995
8	P4	paraphrase	ハルシネーション	LLMが事実ではない情報をもっともらしく生成する現象について説明してください。	入力に続くトークンを予測することで文章を生成し、出力が事実と異なる内容になる。モデルは「ツー...	2.309
9	P5	paraphrase	MCP（Model Context Protocol）	AIアプリと外部ツールやデータソースを共通の方法で接続する仕組みについて説明してください。	LLMの呼び出しを共通の方法で受け入れる設計。外部ツールはJSONで、データソースはDBやA...	5.741
10	U1	unknown	Knowledge Graph	Knowledge Graphとは何ですか？	モデルが参照するデータベース。グラフのedgeをモデルに渡す、と考える。Pydantic A...	1.740
11	U2	unknown	Embedding	Embeddingとは何ですか？	モデルの出力を、特定のフォーマットやデータに合った形で受け取る仕組み。APIで直接出力するの...	3.229
12	U3	unknown	ベクトルデータベース	ベクトルデータベースとは何ですか？	NLPでテキストを数値で表し、検索で同じテキストを再度抽出する。Claude Desktop...	2.507
			Chain-of-		LLMが解決する最難しいうところの処理	

次のステップ: after_df を使用してコードを生成 New interactive sheet

```
from pathlib import Path

GLOSSARY_JSON = Path("/content/glossary.json")

print("exists:", GLOSSARY_JSON.exists())
print(
    "size:",
    GLOSSARY_JSON.stat().st_size if GLOSSARY_JSON.exists() else None
)
```

```
exists: True
size: 31120
```

0	E1	exact	LoRA	か？	ニングにおいて非常に重要な技術の一つ...	小さな行列の積（低ランク行列）で表し、...	16.164	9.784

```
import json

with open("/content/glossary.json", encoding="utf-8") as f:
    glossary_data = json.load(f)

glossary_terms = glossary_data["terms"]

print("glossary terms:", len(glossary_terms))


def search_glossary(question):
    """
    質問文に glossary.json の term が含まれていれば、
    その定義を返す単純な文字列検索。
    見つからなければ None を返す。
    """

    matches = []
```

```
    for item in glossary_terms:
        term = item["term"]

        if term in question:
            matches.append({
                "term": term,
                "definition": item["definition"],
            })

    return matches
```

```
glossary terms: 58
```

6	P2	paraphrase	QLoRA	QLoRAでは、モデルのメモリ使用量を抑えながらどの	QLoRA（Quantized LoRA）は、LoRA（低秩	固定ビット数（例：4ビットでモデルを読み込む）でメモリ	11.307	11.076

```
search_rows = []

for item in TEST:
    matches = search_glossary(item["question"])

    if matches:
        result = "¥n".join(
            f"{m['term']}: {m['definition']}"
            for m in matches
        )
        found = True
    else:
        result = "検索結果なし"
        found = False

    search_rows.append({
        "id": item["id"],
        "category": item["category"],
        "term": item["term"],
        "question": item["question"],
        "search_found": found,
        "search_result": result,
    })

search_df = pd.DataFrame(search_rows)

display(search_df)
```

11	U2	unknown	Embedding	Embeddingとは何ですか？	テキストや画像などの情報を「埋め込み（埋め込む…	受け取る仕組み。APIで直接出力するの…	16.590	3.229
12	U3	unknown	ベクトルデータベース	ベクトルデータベースとは何ですか？	**ベクトルデータベース**（Vector Database）は、データが「ベクトル	NLPでテキストを数値で表し、検索で同じテキストを再度抽出する。Claude	11.245	2.507

	id	category	term	question	search_found	search_result
0	E1	exact	LoRA	LoRAとは何ですか？	True	LoRA: 事前学習済みの重みは固定したまま、重みの更新分を2つの小さな行列の積（低ランク行...
1	E2	exact	QLoRA	QLoRAとは何ですか？	True	LoRA: 事前学習済みの重みは固定したまま、重みの更新分を2つの小さな行列の積（低ランク行...
2	E3	exact	ファインチューニング	ファインチューニングとは何ですか？	True	ファインチューニング: 事前学習済みモデルを、特定のタスクや形式に合ったデータで追加学習させ...
3	E4	exact	ハルシネーション	ハルシネーションとは何ですか？	True	ハルシネーション: LLMが事実と異なる内容や存在しない情報を、もっともらしく生成してしまう...
4	E5	exact	MCP（Model Context Protocol）	MCP（Model Context Protocol）とは何ですか？	True	MCP（Model Context Protocol）: AIアプリと外部のツールやデータソ...
5	P1	paraphrase	LoRA	LoRAは、事前学習済みモデルをどのように効率よく追加学習する仕組みですか？	True	事前学習: 大量の汎用テキストで、次のトークンを予測するようにモデルを学習させる最初の段階。...
6	P2	paraphrase	QLoRA	QLoRAでは、モデルのメモリ使用量を抑えながらどのようにLoRA学習を行いますか？	True	LoRA: 事前学習済みの重みは固定したまま、重みの更新分を2つの小さな行列の積（低ランク行...
7	P3	paraphrase	ファインチューニング	事前学習済みモデルを特定の用途に合わせて調整する方法を説明してください。	True	事前学習: 大量の汎用テキストで、次のトークンを予測するようにモデルを学習させる最初の段階。...
8	P4	paraphrase	ハルシネーション	LLMが事実ではない情報をもっともらしく生成する現象について説明してください。	False	検索結果なし
9	P5	paraphrase	MCP（Model Context Protocol）	AIアプリと外部ツールやデータソースを共通の方法で接続する仕組みについて説明してください。	True	外部ツール: Webページの取得、ファイル操作、データベース、他社のAPIなど、エージェント...
10	U1	unknown	Knowledge Graph	Knowledge Graphとは何ですか？	False	検索結果なし
11	U2	unknown	Embedding	Embeddingとは何ですか？	False	検索結果なし
12	U3	unknown	ベクトルデータベース	ベクトルデータベースとは何ですか？	False	検索結果なし
13	U4	unknown	Chain-of-Thought	Chain-of-Thoughtとは何ですか？	False	検索結果なし
14	U5	unknown	RLHF	RLHFとは何ですか？	False	検索結果なし

次のステップ: search_df を使用してコードを生成 New interactive sheet

```
search_summary = (
    search_df
    .groupby("category")["search_found"]
    .agg(["sum", "count"])
)

search_summary["rate"] = (
    search_summary["sum"] / search_summary["count"]
)

display(search_summary)
```

category	sum	count	rate
exact	5	5	1.0
paraphrase	4	5	0.8
unknown	0	5	0.0

次のステップ: search_summary を使用してコードを生成 New interactive sheet

```
SEARCH_PATH = WORK / "glossary_search_eval.json"

SEARCH_PATH.write_text(
    json.dumps(
        search_rows,
        ensure_ascii=False,
        indent=2,
    ),
    encoding="utf-8",
)
```

```
print("saved:", SEARCH_PATH)
```

```
saved: /content/qlora_lab/glossary_search_eval.json
```

```
# 比較用の代表5問
selected_ids = ["E2", "E4", "P3", "P4", "U5"]

comparison_rows = []

for qid in selected_ids:
    b = next(x for x in before_rows if x["id"] == qid)
    a = next(x for x in after_rows if x["id"] == qid)
    s = next(x for x in search_rows if x["id"] == qid)

    comparison_rows.append({
        "id": qid,
        "category": b["category"],
        "question": b["question"],
        "before": b["answer_before"],
        "after": a["answer_after"],
        "search_found": s["search_found"],
        "search_result": s["search_result"],
    })

representative_df = pd.DataFrame(comparison_rows)

display(representative_df)
```

	id	category	question	before	after	search_found	search_result
0	E2	exact	QLoRAとは何ですか？	**QLoRA** は、**Qwen3.5** という大規模言語モデル（LLM）の一種です。...	ベースモデルの重みを4ビットなどに量子化してメモリを節約したうえで、LoRAで学習する手法。...	True	LoRA: 事前学習済みの重みは固定したまま、重みの更新分を2つの小さな行列の積（低ランク行...
1	E4	exact	ハルシネーションとは何ですか？	**ハルシネーション**（Hulshinēshon）は、物理学、特に**量子力学**において...	LLMが事実と異なる内容や存在しない情報を、もっともらしく生成してしまう現象。モデルが知らな...	True	ハルシネーション: LLMが事実と異なる内容や存在しない情報を、もっともらしく生成してしまう...
2	P3	paraphrase	事前学習済みモデルを特定の用途に合わせて調整する方法を説明してください。	事前学習済みモデル（Pre-trained Models）は、大規模言語モデル（LLM）や画...	AIE2026の演習と宿題で、所轄のモデル（LoRA-100、Pi-5、Pi-1）を使い、そ...	True	事前学習: 大量の汎用テキストで、次のトークンを予測するようにモデルを学習させる最初の段階。...
3	P4	paraphrase	LLMが事実ではない情報をもっともらしく生成する現象について説明してください。	LLM（大規模言語モデル）が事実ではない情報をもっともらしく生成する現象は、**「生成誤解（...	入力に続くトークンを予測することで文章を生成し、出力が事実と異なる内容になる。モデルは「ツー...	False	検索結果なし
				RLHF は、	事前学習済みのモデルを、エージェント		

次のステップ: representative_df を使用してコードを生成 New interactive sheet

4b. 学習が止まった・できなかった場合

途中で止まった（ランタイムの切断など）：チェックポイントは 10 ステップごとに `/content/qlora_lab/ckpt/` に保存されています。ファイルが残っていれば、セクション 0〜3 を実行し直してから、上の学習セルの `trainer.train()` を `trainer.train(resume_from_checkpoint=True)` に書き換えて実行すると、最後のチェックポイントから 40 ステップ（20 エポック）まで続きを学習します。ランタイムごと消えてファイルが無い場合は、学習をやり直すか、下の方法に切り替えます。

学習できなかった（GPU が確保できない、時間内に終わらない）：講師が同じノートブックを T4 で実行した結果（学習後の出力とスコア）を使って、比較と考察を進めます。セクション 5 以降のセルは学習後のモデル `tuned` が無いと動かないので、講師の結果を見ながら説明を読んでください。提出するときは、セクション 7 の `executed_training_myself` を `False` にし、本人が実行できた範囲を明記します。

5. 学習後評価（学習前と同じ test・同じ生成設定）

やること: 学習後のモデル `tuned` で、学習前とまったく同じ 21 問・同じ生成設定で評価し、前後を並べます。変えたのは adapter の有無だけです。

見るところ

- 比較表: `format_rate`（形式遵守率）と `content_rate`（内容正解率）が学習前からどう変わったか。
- `退行`: 学習前は形式も内容も通っていたのに、学習後に落ちた質問です。空のリストが期待値です。
- 質問ごとの表: 残った失敗を読み、どの型かを見分けてください。
  - 根拠なしの質問（正解の `needs_review` が `True`）に、FAQ の別項目を当てはめて断定した。
  - 根拠ありの質問（正解が `False`）に、「記載がありません」と確認要求で逃げた。

学習後の回答は、教師データの文（FAQ の原文）をほぼそのまま返す形になります。形式と文面は揃いますが、「FAQ に根拠があるか」の判断まで学習できたとは限りません。

seed を固定しても、GPU 上の学習は完全には再現しません。形式遵守率と内容正解率の割合はほぼ同じでも、どの質問が残るかは実行ごとに変わることがあります。隣の人と「残った質問」が違っても異常ではないので、失敗の型で比べてください。

```
after_summary, after_rows = evaluate(tuned, "after")
compare = pd.DataFrame([before_summary, after_summary]).set_index("label")[["n", "format_rate", "content_rate", "both_rate", "seconds"]]
display(compare)
# 学習前は形式も内容も通ったのに、学習後に落ちた質問（退行）
regressed = [b["question"] for b, a in zip(before_rows, after_rows) if b["both_ok"] and not a["both_ok"]]
print("退行:", regressed)

# 質問ごとの出力原文。形式（JSON・必須キー・型）と内容（needs_review の一致と必須語句）の判定を自分の目で確かめる
detail = pd.DataFrame([{"質問": b["question"], "正解の needs_review": b["gold_needs_review"],
                        "学習前の出力": b["raw"], "学習前 形式/内容": f"{mark(b['format_ok'])}/{mark(b['content_ok'])}",
                        "学習後の出力": a["raw"], "学習後 形式/内容": f"{mark(a['format_ok'])}/{mark(a['content_ok'])}"}
                       for b, a in zip(before_rows, after_rows)])
display(detail)
```

6. 知識を問う質問（形式を学んでも知識は保証されない）

やること: 形式の実験とは別に、FAQ にも学習データにも無い事実を 3 問尋ねます（対照として FAQ にある質問を 1 問加えます）。望ましい応答は「確認が必要（`needs_review: true`）」です。

応答を 3 つに分類します。

分類	意味
`format_error`	JSON の形式を守れていない（学習前に多い）
`needs_review`	「記載がありません。確認が必要です」と返した（望ましい応答）
`asserted`	`needs_review: false` で言い切った。FAQ に無い質問に対してなら、根拠のない断定（創作）です

見るところ: 学習前は形式違反になりやすく、学習後は「確認が必要」と「断定」の両方が出ることがあります。出力の原文も表に出すので、断定した場合に何を根拠にしたように見えるか（FAQ の別項目を流用しているか）を読んでください。「Fine-Tuning では知識は絶対に直らない」と先に結論にせず、今回のデータでの結果を報告します。